Rapport sur le Codage de Huffman : Compression et Arbre Binaire

Yassir Chakir et Anas Sabir *Équipe EF07, Sciences du Numérique, 2024-2025*

Résumé

Ce rapport présente la conception et l'implémentation d'un programme de codage de Huffman, comprenant la compression de fichiers et la gestion des arbres binaires. Il expose la méthode employée pour répondre aux principaux défis, notamment la construction de l'arbre de Huffman, la représentation des codes binaires et leur intégration dans un format de fichier compressé. Nous détaillons les algorithmes utilisés, les choix techniques réalisés et la démarche adoptée pour valider le système. Enfin, un bilan technique et personnel met en lumière les résultats obtenus et propose des perspectives d'amélioration.

1 Introduction

Le codage de Huffman est une méthode de compression sans perte qui réduit efficacement la taille des fichiers en assignant des codes binaires de longueur variable en fonction des fréquences des caractères. Ce projet a pour objectif de développer une application capable de compresser un fichier texte tout en conservant la possibilité de le décompresser ultérieurement.

Le rapport est structuré comme suit : la section 2 présente l'architecture en modules, la section 3 détaille les choix techniques effectués, la section 4 explique les principaux algorithmes, la section 5 discute des tests réalisés, et la section 6 aborde les difficultés rencontrées. Les sections 7 et 8 concluent avec un bilan technique et personnel.

2 Architecture modulaire

Le programme repose sur deux modules principaux interconnectés :

- **Module Compression :** Responsable de la compression des fichiers en utilisant les codes binaires générés par l'arbre de Huffman.
- **Module Arbre Binaire :** Permet de construire et de gérer l'arbre de Huffman, en triant les caractères par fréquence pour générer les codes binaires.

Ces modules interagissent pour produire un fichier compressé et assurer la traçabilité des données nécessaires à la décompression.

3 Principaux choix réalisés

- **Représentation de l'arbre :** Utilisation d'un arbre binaire parfait pour stocker les fréquences des caractères et leurs codes.

— **Constructions des noeuds sans tri séparé :** Plutôt que d'utiliser un module de tri séparé, nous avons choisi de trouver directement les deux plus petites fréquences et de les permuter avec les deux derniers symboles. Cette méthode réduit considérablement la complexité tout en assurant la construction correcte de l'arbre de Huffman.
— **Format des fichiers compressés :** Intégration des métadonnées (fréquences et structure de l'arbre) en début de fichier pour permettre la décompression.
— **Gestion des données binaires :** Les codes sont regroupés par blocs de 8 bits pour être enregistrés sous forme d'octets.

4 Algorithmes

4.1 Construction de l'arbre de Huffman

L'arbre est construit en fusionnant les deux nœuds ayant les fréquences les plus faibles. Ce processus est répété jusqu'à obtenir un seul arbre racine.

Algorithm 1: Construction de l'arbre de Huffman

Input: Table des fréquences avec les symboles et leurs fréquences
Output: Racine de l'arbre de Huffman

```
while la taille de la table des fréquences contient plus d'un élément do
    Permuter les deux derniers éléments de la table avec les deux éléments ayant
      les plus petites fréquences;
    Fusionner les deux éléments ayant les plus petites fréquences;
    Créer un nouveau nœud avec une fréquence égale à la somme des fréquences
      des deux éléments fusionnés;
    Remplacer ces deux éléments par le nouveau nœud dans la table;
    Réduire la taille de la table des fréquences d'un élément;
```

4.2 Génération des codes binaires

Chaque caractère est associé à un code binaire via un parcours de l'arbre :
— Une descente à gauche ajoute un `0` au code.
— Une descente à droite ajoute un `1`.

4.3 Encodage du fichier

Le texte est transformé en une séquence de bits en remplaçant chaque caractère par son code binaire. Les bits sont ensuite regroupés en octets pour former le fichier compressé.

5 Types de données

Les types utilisés dans l'implémentation sont définis comme suit :

5.1 Type T_Element

Ce type représente un élément de base, constitué d'un caractère `Cle` et de sa fréquence d'apparition `Frequence` dans le texte. Ce type est utilisé pour stocker les informations nécessaires à la construction de l'arbre de Huffman.

```
type T_Element is
    record
        Cle : Character;
        Frequence : Integer;
    end record;
```

5.2 Type T_Tab

Il s'agit d'un tableau de 256 éléments de type `T_Element`, utilisé pour représenter les fréquences des 256 caractères possibles dans un fichier texte. Ce tableau est essentiel pour la construction de l'arbre de Huffman.

```
type T_Tab is array(1..256) of T_Element;
```

5.3 Type T_Noeud

Ce type définit un nœud de l'arbre de Huffman. Chaque nœud contient un caractère `Cle`, une valeur `Valeur` qui est la somme des fréquences des nœuds enfants, ainsi que deux pointeurs vers les sous-arbres gauche et droit.

```
type T_Noeud is
    record
        Cle : Character;
        Valeur : Integer;
        Sous_Arbre_Gauche : T_ABR;
        Sous_Arbre_Droit : T_ABR;
    end record;
```

5.4 Type T_ABR

Ce type est un pointeur qui permet de manipuler dynamiquement les nœuds de l'arbre binaire. Il est utilisé pour accéder aux nœuds de l'arbre de Huffman.

```
type T_ABR is access T_Noeud;
```

6 Démarche de test

Pour garantir la fiabilité, plusieurs niveaux de tests ont été effectués :

- **Tests unitaires :** Validation individuelle des modules de compression et d'arbre binaire.
- **Tests d'intégration :** Vérification de la compatibilité entre les modules en compressant des fichiers variés.
- **Cas limites :** Évaluation avec des fichiers contenant peu ou beaucoup de caractères uniques.

7 Difficultés rencontrées et solutions

- **Remarque :** Suite à notre mauvaise lecture de sujet au début, on n'a pas constater l'introduction de symbole ' $ ' qui représente la fin du fichier texte . Il était impossible d'introduire ce symbole après l'écriture de 80% du programme.
- **Création de l'arbre : gestion des symboles déjà traités :** Une difficulté initiale concernait la répétition des symboles dans l'arbre de Huffman. Les symboles déjà traités réapparaissaient dans la structure, ce qui perturbait la construction correcte de l'arbre. **Solution** : Exclure explicitement les éléments déjà utilisés en les déplaçant vers la fin du tableau des fréquences, tout en réduisant la taille effective de ce tableau à chaque itération. Cela a permis de limiter les opérations uniquement aux éléments restants.
- **Complexité de l'affichage de l'arbre :** Lors de l'implémentation pour l'option "bavard", une difficulté majeure résidait dans le respect de la forme d'affichage demandée pour représenter l'arbre de Huffman. Cette option exige une sortie détaillée et lisible montrant les étapes successives de la construction de l'arbre, ce qui s'est avéré complexe à gérer.

8 Organisation de l'équipe

La répartition des tâches entre les membres de l'équipe est la suivante :

Membre	Tâches
Sabir Anas	Implémentation du module Arbre Binaire, génération des codes, gestion des tests et écriture du rapport
Yassir Chakir	Développement du module Compression et gestion des tests

Table 1 – Répartition des tâches au sein de l'équipe

9 Bilan technique et perspectives

Le projet a atteint ses objectifs principaux, mais des améliorations peuvent être envisagées :

- **Optimisation de la gestion des données :** Réduction de la taille des métadonnées dans le fichier compressé.
- **Performance :** Utilisation d'une structure de tas pour accélérer la construction de l'arbre.

10 Bilan personnel

10.1 Yassir Chakir

J'ai apprécié travailler sur ce projet qui m'a permis de renforcer mes compétences en structures de données.

10.2 Anas Sabir

Ce projet a été une expérience très enrichissante, en particulier dans la gestion des fichiers binaires et le travail sur des structures de données complexes, telles que les arbres.